Machine learning Lab Observation

ITA0614 - Machine Learning

Experiment 1

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Name : Variganji Sumanth Babu

Reg.No : 192524447

Problem 1: Enjoy sport

Aim

To implement the FIND-S algorithm in Python on the EnjoySport training data stored in a list and obtain the most specific hypothesis.

Algorithm

1. Initialise h to the most specific hypothesis: h = <0, 0, ..., 0>.
2. For each training example x:
   (a) If x is a negative example, ignore it.
   (b) If x is positive, examine each attribute constraint in h.
   If h[i] = 0, set h[i] to x[i]; if h[i] differs from x[i], set h[i] to "?".
3. Output the final hypothesis h.

Program:

find_s_a.py - Python 3

```python
# Program 1: FIND-S on EnjoySport data stored in a list
attributes = ["Sky", "AirTemp", "Humidity", "Wind", "Water", "Forecast"]
training_data = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"],
]

hypothesis = ["0"] * len(attributes)
print("Initial hypothesis:", hypothesis)
for step, example in enumerate(training_data, start=1):
    x, label = example[:-1], example[-1]
    if label == "Yes":
        for i, value in enumerate(x):
            if hypothesis[i] == "0":
                hypothesis[i] = value
            elif hypothesis[i] != value:
                hypothesis[i] = "?"
    print(f"After example {step} ({label}): {hypothesis}")
print("Final (most specific) hypothesis:", hypothesis)
```

Input

EnjoySport data set: 4 training examples, 6 attributes (Sky, AirTemp, Humidity, Wind, Water, Forecast) and target concept EnjoySport (Yes = positive, No = negative). The data is hard-coded as a list of lists.

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Output

Terminal - Output

```
$ python find_s_a.py
Initial hypothesis: ['0', '0', '0', '0', '0', '0']
After example 1 (Yes): ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
After example 2 (Yes): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After example 3 (No): ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
After example 4 (Yes): ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Final (most specific) hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result

The FIND-S algorithm was implemented and executed successfully. The most specific hypothesis consistent with the positive examples is <Sunny, Warm, ?, Strong, ?, ?>. The negative example did not change the hypothesis.

Problem 2: Play Cricket

Aim

To implement FIND-S by reading training examples from a CSV file using the csv module of Python.

Algorithm

1. Initialise h to the most specific hypothesis: h = <0, 0, ..., 0>.
2. For each training example x:
   (a) If x is a negative example, ignore it.
   (b) If x is positive, examine each attribute constraint in h.
   If h[i] = 0, set h[i] to x[i]; if h[i] differs from x[i], set h[i] to "?".
3. Output the final hypothesis h.

Program:

find_s_b.py - Python 3

```python
# Program 2: FIND-S using the csv module and a CSV training file
import csv
from pathlib import Path

csv_file = Path("play_cricket.csv")
csv_content = """Outlook,Temperature,Humidity,Wind,PlayCricket
Sunny,Mild,Normal,Weak,Yes
Rainy,Mild,Normal,Weak,No
Sunny,Hot,Normal,Weak,Yes
Sunny,Hot,High,Strong,No
Sunny,Cool,High,Weak,Yes
"""
if not csv_file.exists():
    csv_file.write_text(csv_content, encoding="utf-8")

with csv_file.open(newline="", encoding="utf-8") as file:
    reader = csv.reader(file)
    header = next(reader)
    data = list(reader)

attributes = header[:-1]
hypothesis = ["0"] * len(attributes)
print("Attributes:", attributes)
print("Target concept:", header[-1])
print("No. of samples:", len(data))
print("Initial hypothesis:", hypothesis)
for step, row in enumerate(data, start=1):
    x, label = row[:-1], row[-1].strip()
    if label.lower() == "yes":
        for i, value in enumerate(x):
            if hypothesis[i] == "0":
                hypothesis[i] = value
            elif hypothesis[i] != value:
                hypothesis[i] = "?"
    print(f"Step {step} ({label}): h = {hypothesis}")
print("Most specific hypothesis:", hypothesis)
```

Input

PlayCricket data set (play_cricket.csv): 5 examples, 4 attributes (Outlook, Temperature, Humidity, Wind) and target concept PlayCricket (Yes / No). The first row stores the attribute names. The program creates the CSV file with these records if it is not already present, then reads it.

Outlook	Temperature	Humidity	Wind	PlayCricket
Sunny	Mild	Normal	Weak	Yes
Rainy	Mild	Normal	Weak	No
Sunny	Hot	Normal	Weak	Yes
Sunny	Hot	High	Strong	No
Sunny	Cool	High	Weak	Yes

Output

Terminal - Output

```
$ python find_s_b.py
Attributes: ['Outlook', 'Temperature', 'Humidity', 'Wind']
Target concept: PlayCricket
No. of samples: 5
Initial hypothesis: ['0', '0', '0', '0']
Step 1 (Yes): h = ['Sunny', 'Mild', 'Normal', 'Weak']
Step 2 (No): h = ['Sunny', 'Mild', 'Normal', 'Weak']
Step 3 (Yes): h = ['Sunny', '?', 'Normal', 'Weak']
Step 4 (No): h = ['Sunny', '?', 'Normal', 'Weak']
Step 5 (Yes): h = ['Sunny', '?', '?', 'Weak']
Most specific hypothesis: ['Sunny', '?', '?', 'Weak']
```

Result

The FIND-S algorithm was executed on the CSV training data. The most specific hypothesis is <Sunny, ?, ?, Weak>; in this sample, Outlook must be Sunny and Wind must be Weak, while Temperature and Humidity can take any value.

Problem 3: Buy Laptop

Aim

To implement FIND-S using pandas DataFrame operations and learn the most specific hypothesis for the Buy Laptop data.

Algorithm

1. Initialise h to the most specific hypothesis: h = <0, 0, ..., 0>.
2. For each training example x:
   (a) If x is a negative example, ignore it.
   (b) If x is positive, examine each attribute constraint in h.
   If h[i] = 0, set h[i] to x[i]; if h[i] differs from x[i], set h[i] to "?".
3. Output the final hypothesis h.

Program:

find_s_c.py - Python 3

```python
# Program 3: FIND-S using pandas DataFrame operations
from pathlib import Path
import pandas as pd

csv_file = Path("buy_laptop.csv")
csv_content = """Brand,RAM,Storage,Price,Buy
Dell,8GB,SSD,Low,Yes
HP,4GB,HDD,High,No
Dell,8GB,SSD,Medium,Yes
Dell,4GB,HDD,High,No
Dell,16GB,SSD,Medium,Yes
Lenovo,8GB,SSD,Low,No
"""
if not csv_file.exists():
    csv_file.write_text(csv_content, encoding="utf-8")

data = pd.read_csv(csv_file)
attributes = list(data.columns[:-1])
target = data.columns[-1]
positive = data[data[target].str.lower() == "yes"]
negative = data[data[target].str.lower() == "no"]
print("Training data set:")
print(data.to_string(index=False))
print("Positive examples:", len(positive))
print("Negative examples:", len(negative))

hypothesis = positive.iloc[0][attributes].astype(str).tolist()
print("h0 =", hypothesis)
for step, (_, row) in enumerate(positive.iloc[1:].iterrows(), start=1):
    example = row[attributes].astype(str).tolist()
    hypothesis = [a if a == b else "?" for a, b in zip(hypothesis, example)]
    print(f"h{step} =", hypothesis)
print("Most specific hypothesis:", dict(zip(attributes, hypothesis)))
```

Input

Buy Laptop data set (buy_laptop.csv): 6 examples, 4 attributes (Brand, RAM, Storage, Price) and target concept Buy (Yes / No), loaded into a pandas DataFrame. The program creates the CSV file if it is not already present.

Brand	RAM	Storage	Price	Buy
Dell	8GB	SSD	Low	Yes
HP	4GB	HDD	High	No
Dell	8GB	SSD	Medium	Yes
Dell	4GB	HDD	High	No
Dell	16GB	SSD	Medium	Yes
Lenovo	8GB	SSD	Low	No

Output

Terminal - Output

```
$ python find_s_c.py
Training data set:
 Brand  RAM Storage  Price Buy
  Dell  8GB     SSD    Low Yes
    HP  4GB     HDD   High  No
  Dell  8GB     SSD Medium Yes
  Dell  4GB     HDD   High  No
  Dell 16GB     SSD Medium Yes
Lenovo  8GB     SSD    Low  No
Positive examples: 3
Negative examples: 3
h0 = ['Dell', '8GB', 'SSD', 'Low']
h1 = ['Dell', '8GB', 'SSD', '?']
h2 = ['Dell', '?', 'SSD', '?']
Most specific hypothesis: {'Brand': 'Dell', 'RAM': '?', 'Storage': 'SSD', 'Price': '?'}
```

Result

FIND-S was implemented using pandas. The most specific hypothesis is <Dell, ?, SSD, ?>. For these training examples, the positive cases share the Dell brand and SSD storage; RAM and Price vary.

Problem 4: Mushroom

Aim

To implement FIND-S using user-defined functions, display the hypothesis after every training example, and classify new samples.

Algorithm

1. Initialise h to the most specific hypothesis: h = <0, 0, ..., 0>.
2. For each training example x:
   (a) If x is a negative example, ignore it.
   (b) If x is positive, examine each attribute constraint in h.
   If h[i] = 0, set h[i] to x[i]; if h[i] differs from x[i], set h[i] to "?".
3. Output the final hypothesis h.

Program:

find_s_d.py - Python 3

```python
# Program 4: FIND-S with functions, trace history, and new-sample classification
attributes = ["Colour", "Size", "Shape", "Surface", "Odour"]
mushrooms = [
    ["Brown", "Small", "Convex", "Smooth", "None", "Yes"],
    ["Brown", "Large", "Convex", "Smooth", "None", "Yes"],
    ["Red", "Small", "Convex", "Scaly", "Foul", "No"],
    ["Brown", "Small", "Flat", "Smooth", "None", "Yes"],
    ["Brown", "Large", "Bell", "Scaly", "Foul", "No"],
]

def find_s(examples):
    hypothesis = ["0"] * (len(examples[0]) - 1)
    history = []
    for step, row in enumerate(examples, start=1):
        x, label = row[:-1], row[-1]
        if label == "Yes":
            for i, value in enumerate(x):
                if hypothesis[i] == "0":
                    hypothesis[i] = value
                elif hypothesis[i] != value:
                    hypothesis[i] = "?"
        history.append((step, label, hypothesis.copy()))
    return hypothesis, history

def classify(hypothesis, sample):
    return "Yes" if all(a == "?" or a == b for a, b in zip(hypothesis, sample)) else "No"

hypothesis, history = find_s(mushrooms)
print(f"{'Step':<6}{'Class':<7}Hypothesis")
for step, label, current in history:
    print(f"{step:<6}{label:<7}{current}")
print("Final hypothesis:", hypothesis)
new_samples = [
    ["Brown", "Large", "Flat", "Smooth", "None"],
    ["Red", "Small", "Convex", "Smooth", "None"],
]
for sample in new_samples:
    print("New sample", sample, "->", classify(hypothesis, sample))
```

Input

Mushroom data set: 5 examples, 5 attributes (Colour, Size, Shape, Surface, Odour) and target concept Edible (Yes / No). The records are stored in the program.

Colour	Size	Shape	Surface	Odour	Edible
Brown	Small	Convex	Smooth	None	Yes
Brown	Large	Convex	Smooth	None	Yes
Red	Small	Convex	Scaly	Foul	No
Brown	Small	Flat	Smooth	None	Yes
Brown	Large	Bell	Scaly	Foul	No

Output

Terminal - Output

```
$ python find_s_d.py
Step  Class  Hypothesis
1     Yes    ['Brown', 'Small', 'Convex', 'Smooth', 'None']
2     Yes    ['Brown', '?', 'Convex', 'Smooth', 'None']
3     No     ['Brown', '?', 'Convex', 'Smooth', 'None']
4     Yes    ['Brown', '?', '?', 'Smooth', 'None']
5     No     ['Brown', '?', '?', 'Smooth', 'None']
Final hypothesis: ['Brown', '?', '?', 'Smooth', 'None']
New sample ['Brown', 'Large', 'Flat', 'Smooth', 'None'] -> Yes
New sample ['Red', 'Small', 'Convex', 'Smooth', 'None'] -> No
```

Result

The most specific hypothesis is <Brown, ?, ?, Smooth, None>. The sample (Brown, Large, Flat, Smooth, None) is classified as Edible = Yes, while (Red, Small, Convex, Smooth, None) is classified as Edible = No.

Problem 5: Bank loan approval

Aim

To implement FIND-S using NumPy arrays and verify that the learned hypothesis covers all positive examples and no negative examples in the loan data.

Algorithm

1. Initialise h to the most specific hypothesis: h = <0, 0, ..., 0>.
2. For each training example x:
   (a) If x is a negative example, ignore it.
   (b) If x is positive, examine each attribute constraint in h.
   If h[i] = 0, set h[i] to x[i]; if h[i] differs from x[i], set h[i] to "?".
3. Output the final hypothesis h.

Program:

find_s_e.py - Python 3

```python
# Program 5: FIND-S using NumPy arrays and hypothesis verification
import numpy as np

data = np.array([
    ["High", "Salaried", "Good", "Adult", "Yes"],
    ["High", "Salaried", "Good", "Senior", "Yes"],
    ["Low", "Self", "Bad", "Adult", "No"],
    ["Medium", "Salaried", "Good", "Adult", "Yes"],
    ["High", "Self", "Bad", "Adult", "No"],
    ["Medium", "Salaried", "Good", "Young", "Yes"],
], dtype=str)
attributes = ["Income", "Employment", "CreditHistory", "Age"]
X, y = data[:, :-1], data[:, -1]
positive, negative = X[y == "Yes"], X[y == "No"]

print("Number of positive examples:", len(positive))
print("Number of negative examples:", len(negative))
hypothesis = positive[0].copy()
print("h0 =", hypothesis.tolist())
for step, example in enumerate(positive[1:], start=1):
    hypothesis = np.where(hypothesis == example, hypothesis, "?")
    print(f"h{step} =", hypothesis.tolist())

def covers(h, x):
    return bool(np.all((h == "?") | (h == x)))

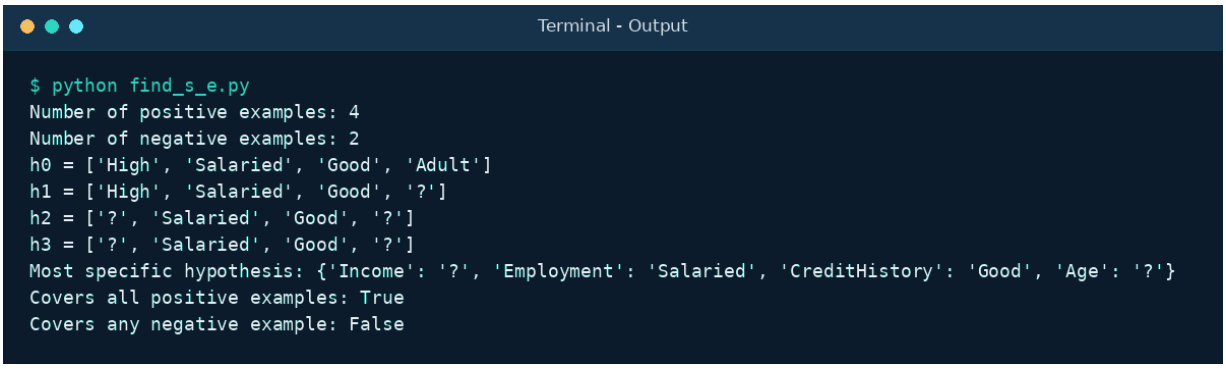
print("Most specific hypothesis:", dict(zip(attributes, hypothesis.tolist())))
print("Covers all positive examples:", all(covers(hypothesis, x) for x in positive))
print("Covers any negative example:", any(covers(hypothesis, x) for x in negative))
```

Input

Bank Loan data set: 6 examples, 4 attributes (Income, Employment, CreditHistory, Age) and target concept Approved (Yes / No), stored as a NumPy string array.

Income	Employment	CreditHistory	Age	Approved
High	Salaried	Good	Adult	Yes
High	Salaried	Good	Senior	Yes
Low	Self	Bad	Adult	No
Medium	Salaried	Good	Adult	Yes
High	Self	Bad	Adult	No
Medium	Salaried	Good	Young	Yes

Output



Result

The most specific hypothesis is <?, Salaried, Good, ?>. Verification returns True for covering all positive examples and False for covering any negative example. In this dataset, the positive examples share Salaried employment and a Good credit history.